Container Deep Dive: BESContainer, Its Subclasses, and Container Storage

Scope

This document covers:

- `BESContainer` and every subclass outside `retired/`:
  - `BESFileContainer`
  - `ngap::NgapOwnedContainer`
  - `S3Container`, `GatewayContainer`, `CmrContainer`, `HttpdCatalogContainer`
- The container stores:
  - the base: `BESContainerStorage`
  - the in-memory store: `BESContainerStorageVolatile`
  - the stores built on it: `BESFileContainerStorage`, `NgapOwnedContainerStorage`, and the module stores
  - the test-only `BESContainerStorageFile`
- The `BESContainerStorageList` singleton.
- The framework code that creates, looks up, uses, and releases containers.

The two classes the request named, `NgapOwnedContainer` and `BESFileContainer`, are covered in the most detail.

Each finding is labeled with how it was checked:

- **Verified**: checked with a live test, or with a direct code trace that I read line by line.
- **Inferred**: follows from reading the code, but not tested.

All line numbers are for `master` at `9b4462e99`.

Summary

What a container is

A container is a small, per-request handle for one dataset. It has three names and a type:

- **Symbolic name**: what the client calls the dataset.
- **Real name**: what the BES actually opens, such as a full file path, a URL, or an NGAP REST path.
- **Relative name**: the path relative to the data root, before the root was prepended.
- **Type**: the handler that reads the dataset, such as `h5`, `nc` or `dmrpp`.

It also carries the per-request constraint, the DAP4 constraint, the DAP4 function, and an "attributes" string.

The two virtual methods do the real work:

- `access()` makes the data readable and returns a name the handler can open.
- `release()` undoes whatever `access()` did.

How the pieces fit

Containers are made by **stores** (`BESContainerStorage`). Stores are kept, in registration order, in the `BESContainerStorageList` singleton.

A request uses a container like this:

1. `setContainer` asks a named store to build a **prototype** container.
2. `define` asks for that symbolic name. It gets back a **duplicate** of the prototype (`ptr_duplicate()`), sets the constraint on it, and hands it to a `BESDefine`, which owns it.
3. `get` borrows the `BESDefine`'s containers.
4. `BESRequestHandlerList::execute_current()` calls `access()` and picks the handler from `get_container_type()`.
5. The handler usually calls `access()` a second time to get the file name.
6. `BESInterface::end_request()` calls `release()`.

The framework design works, with three structural weak points

1. The **type is only known after `access()`**. `execute_current()` must call `access()` before it can pick a handler, and the handler then calls `access()` again. For remote containers, every subclass has to make `access()` cheap to repeat. The NGAP container avoids the problem by making `access()` a no-op and adding a non-virtual `alt_access()` that the dmrpp handler finds with `dynamic_cast`.
2. **Stores build containers.** That is why `setContainer` carries a store name (`space`) and why every remote data source has its own store subclass whose only job is to call `new XContainer`. A `FIXME` in `BESSetContainerResponseHandler.cc:88-90` says the same thing.
3. **Ownership is spread across five places and documented wrongly in two of them.**
   - The store owns the prototypes.
   - `BESDefine` owns the duplicates.
   - The DHI only borrows them.
   - `BESContainerStorageList` owns the stores through `unique_ptr`, but its doc comment says it does not.
   - `release()` is called only for the containers in the last command's DHI.

Most significant findings

1. **The NGAP container leaks one file descriptor per `setContainer` when the DMR++ cache is on** (Verified). The cache is on by default in `ngap.conf`.
   - The `NgapOwnedContainer(sym, real, type)` constructor calls `FileCache::initialize()` every time.
   - `initialize()` opens a new cache-info file descriptor and overwrites the old one without closing it.
   - Every NGAP request sends `setContainer`, so a long-lived `beslistener` child loses one descriptor per request until it hits `RLIMIT_NOFILE`.
   - A scratch probe that called `initialize()` 50 times went from 3 open descriptors to 53.
2. **Two remote containers can fetch from hosts that are not on `AllowedHosts`** (Verified by code trace).
   - **S3**: `S3Container` builds a *trusted* `http::url` from the client-supplied real name.
   - **NGAP**: for `source/<host>/...` paths, `NgapOwnedContainer` uses `curl::http_get(string, ...)`, which has no AllowedHosts check at all.
   - Both then inject `dmrpp:trust="true"` into the fetched DMR++, which also turns off AllowedHosts for the data reads that DMR++ drives.
   - The NGAP path is gated by `NGAP.UseOPeNDAPBucket`. That key defaults to **true** in the code but is set to **false** in the shipped `ngap.conf`.
3. **The NGAP constructor keeps re-reading BES keys into static members.** The file-cache size compounds if the `.MB` keys are absent (Verified by code trace).
   - It stores `MEGABYTE * read_ulong_key(key, <current static>)`.
   - The second time a container is built with the key absent, the size is about 1.1e16 bytes; the third time, it overflows.
   - The shipped `ngap.conf` sets both keys, so this is latent.
4. **`_duplicate` checks the wrong object in `GatewayContainer`, `CmrContainer` and `HttpdCatalogContainer`** (Verified for Gateway; Inferred for the other two, which use the same code). Each tests `copy_to.d_remoteResource`, which is always null, and then copies a raw owning pointer. A duplicate made after `access()` would double-free. This is latent, because only never-accessed prototypes are duplicated today.
5. **`BESContainerStorageVolatile::isData()` leaks a container on every call** (Verified by code trace). It does not delete the result of `look_for()`, which is a heap copy. Every store except `BESFileContainerStorage` inherits this method.
6. **`BESFileContainerStorage::add_container()` has undefined behavior for an empty real name** (Verified by code trace). The trailing-slash loop indexes `real_name[npos]`.
7. **The `source/` branch indexes past the end of an array for the real name `source/`** (Verified by code trace). `NgapOwnedContainer::build_dmrpp_url_to_owned_bucket()` reads `parts[1]` after a `split()` that drops empty tokens.

Class map

```
BESObj
├── BESContainer                          (dispatch/BESContainer.h)       abstract
│   ├── BESFileContainer                  (dispatch/)                     local file, uncompress cache
│   ├── ngap::NgapOwnedContainer          (modules/dmrpp_module/ngap_container/)  DMR++ text from S3/CMR
│   ├── S3Container                       (modules/s3_reader/)            DMR++ side-car file from a URL
│   ├── GatewayContainer                  (modules/gateway_module/)       any remote file, AllowedHosts
│   ├── CmrContainer                      (modules/cmr_module/)           granule found through CMR
│   └── HttpdCatalogContainer             (modules/httpd_catalog_module/) file from a scraped httpd listing
│
├── BESContainerStorage                   (dispatch/BESContainerStorage.h) abstract
│   ├── BESContainerStorageVolatile       in-memory map<sym, BESContainer*>; makes BESFileContainers
│   │   ├── BESFileContainerStorage       "catalog": include/exclude + TypeMatch from a BESCatalog
│   │   ├── ngap::NgapOwnedContainerStorage   "ngap"
│   │   ├── S3ContainerStorage            "s3"
│   │   ├── GatewayContainerStorage       "gateway"
│   │   ├── CmrContainerStorage           "CMR" (only if CMR.UseContainerAccess=true)
│   │   └── HttpdCatalogContainerStorage  "RemoteResources"
│   └── BESContainerStorageFile           reads "sym real type" lines from a file; unit tests only
│
└── BESContainerStorageList               singleton; ordered list of stores with ref counts
```

BESContainer (dispatch/BESContainer.h)

Data members

All the data members are strings (BESContainer.h:68-77):

Member	Set by	Used by
`d_symbolic_name`	the store, when it builds the container	the store map key; `define`; logging
`d_real_name`	the store (`BESFileContainerStorage` prepends the catalog root)	`access()`; `BESDapTransmit`; DMR++ cache keys
`d_relative_name`	`BESContainerStorageVolatile::add_container` (the path the client sent)	handlers that need the catalog-relative path
`d_container_type`	the store or the constructor; changed by `access()` in some subclasses and by `dap/` when the MDS holds a DMR++	`BESRequestHandlerList::find_handler()`

Member	Set by	Used by
`d_constraint`, `d_dap4_constraint`, `d_dap4_function`	`BESXMLDefineCommand::prep_request`	`BESDapResponse`, `BESConstraintFuncs`
`d_attributes`	`define`'s `<container><attributes>`; `NgapOwnedContainer::alt_access()` (`"as-string"`); `dap/` response handlers (`MDS_HAS_DMRPP`)	`DmrppRequestHandler` (NGAP path); `showDefinitions`

Virtual interface

Method	Contract
`ptr_duplicate()`	Returns a new heap copy. Stores call it from `look_for()`, so every request works on a copy of the store's prototype.
`_duplicate(BESContainer&)`	Copies this object's fields *into* the argument. It is the reverse of a copy constructor, as the comment at `:80-81` notes. The base version copies all eight strings.
`access()`	Makes the resource usable and returns a name the handler opens. It may change `d_container_type`.
`release()`	Undoes `access()`.

Copy and move constructors are defaulted. Nothing in the framework uses them; duplication always goes through `ptr_duplicate()`.

BESFileContainer (dispatch/BESFileContainer.{h,cc})

This is the container for files under a catalog root. It is the only kind of container that `BESContainerStorageVolatile::add_container(sym, real, type)` builds.

Constructor (BESFileContainer.cc:59-66)

The constructor throws `BESForbiddenError` if `real_name` contains `..` anywhere. That check is broader than the message says (`'../' not allowed`): a file named `a..b.nc` is rejected too.

The constructor is not the real path-safety check. By the time the store constructs the container, `BESUtil::check_path()` has already validated the path against the root directory and the symlink policy (BESContainerStorageVolatile.cc:151).

`access()` (:92-112)

`access()` calls `BESUncompressManager3::uncompress(real_name, _target, cache)`.

- **The file is not compressed** (its extension does not match a registered decompressor): the call returns false, and `access()` returns `real_name`.
- **The file is compressed:**
  - If it is already in the uncompress cache, `uncompress()` takes a shared read lock on the cached copy.
  - If not, it creates the file, decompresses it, and downgrades the exclusive lock to a shared one.
  - Either way it returns true. `access()` then sets `_cached = true` and returns the cache file name.

`access()` does not change the container type. The comment in `BESRequestHandlerList::execute_current()` (:235-238) says that decompressing changes the extension so that `get_container_type()` works. That is not what happens.

- The type comes from TypeMatch on the *original* name, in `BESFileContainerStorage::add_container`.
- So the TypeMatch regex has to match names like `foo.h5.gz`.

`release()` (:122-128)

If `_cached` is set, `release()` calls `BESUncompressCache::unlock_and_close(_target)`.

`unlock_and_close()` releases **every** descriptor this process holds on that file (BESFileLockingCache.cc:729-740). Calling `access()` twice (the framework call plus the handler call) takes two read locks, and one `release()` drops both. That means:

- the double `access()` does not leak locks (Verified by code trace);
- but two containers in one request that point at the same compressed file share their locks, so releasing one drops the other's lock too (Inferred; low impact).

Duplication

`ptr_duplicate()` uses the private default constructor and copies only the base fields. `_cached` and `_target` are not copied. That is correct for an unaccessed prototype.

`BESFileContainer::_duplicate` is an override that only calls the base. The destructor does not call `release()`; there is a TODO about that at `.h:75` and `.cc:119`.

ngap::NgapOwnedContainer (modules/dmrpp_module/ngap_container/)

What it is for

This container represents a NASA granule addressed by an NGAP "REST path", for example `collections/C2532426483-ORNL_CLOUD/granules/Daymet_...nc`. Its job is to get the **DMR++ text** for that granule. The dmrpp handler then reads the data straight from S3 using the DMR++.

The container never touches the granule itself.

Construction (NgapOwnedContainer.cc:123-179)

`NgapOwnedContainerStorage::add_container()` calls `new NgapOwnedContainer(sym, real, type)` and hands it to the volatile base. The constructor:

- **ignores `type`** and sets `"owned-ngap"`. No handler is registered under that name; `access()` replaces it with `"dmrpp"` before the framework looks it up;
- stores `d_ngap_path = real_name`. Only `_duplicate` reads it; all the other code uses `get_real_name()`;
- reads these keys into **static** members **on every call**: `NGAP.UseOPeNDAPBucket`, `NGAP.inject_data_urls`, `NGAP.EnableDMRppLocalFilesForTesting` (plus `BES.Catalog.catalog.RootDirectory` when that is on), `NGAP.UseCMRCache` and its size and purge keys, `NGAP.UseDMRppCache` and its size and purge keys, and `NGAP.DMRppFileCacheSize.MB`, `NGAP.DMRppFileCachePurge.MB` and `NGAP.DMRppFileCacheDir`;
- calls `initialize()` again on the two static `MemoryCache`s and the static `FileCache`, and calls `mkdir_p` on the cache directory.

`ptr_duplicate()` uses the default constructor plus `_duplicate()`, which copies the base fields and `d_ngap_path`. So the expensive constructor runs once per `setContainer`, not once per `define`.

Problems in the constructor

1. **File-descriptor leak, one per `setContainer`** (Verified).
   - `FileCache::initialize()` calls `open_cache_info()` (FileCache.h:246-261), which assigns a fresh `open()` result to `d_cache_info_fd` without closing the old one.
   - The leaked descriptors are plain open files with no locks on them, so the only harm is that descriptors run out.
   - A scratch program that called `FileCache::initialize()` 50 times on one object went from 3 to 53 open descriptors.
   - Every Hyrax NGAP request carries a `setContainer ... space="ngap"`; see `modules/dmrpp_module/tests/return_as_dmrpp/dmrpp-return-as-test.bescmd`. The OLFS reuses BES connections, so a single `beslistener` child gains one descriptor per request.
   - **Fix:** initialize the caches once, in `DmrppModule::initialize()` or with a static once-flag, or make `FileCache::initialize()` close any descriptor it already holds.
2. **Compounding file-cache size** (Verified by code trace).
   - Lines 161-166 compute `MEGABYTE * read_ulong_key(KEY, d_dmrpp_file_cache_size_mb)`.
   - When the key is absent, the default passed in is the byte value from the previous construction, which then gets multiplied again.
   - Separately, `read_ulong_key` parses into an `int` (TheBESKeys.cc:411).
   - The shipped `ngap.conf` sets both `.MB` keys, so this is latent.
   - The member names end in `_mb` but the members hold bytes.
3. **The comments do not match the code.**
   - The class doc at `.h:55-59` says `access()` returns the DMR++. It returns `""`.
   - `.h:66` calls `d_enable_dmrpp_local_files_for_testing` a "per instance flag". It is static.
   - The commented-out keys in `ngap.conf.in` (`NGAP.DMRppCacheSize`, `NGAP.CMRCacheSize`, ...) do not match the keys the code reads, which end in `.Items`. Uncommenting them has no effect.

`access()` and `alt_access()`

`access()` (:697-703) only sets the type to `"dmrpp"` and returns `""`. That is enough for `execute_current()` to pick `DmrppRequestHandler`, and it stays cheap to call twice. The TODO at `BESRequestHandlerList.cc:239-242` explains the reason.

`alt_access()` (:714-730) does the real work. Exactly two callers reach it through `dynamic_cast`:

- `DmrppRequestHandler::get_dmrpp_from_container_or_cache()` (DmrppRequestHandler.cc:303-313);
- `ReturnAsDmrppTransmitter::send_dmrpp()` (ReturnAsDmrppTransmitter.cc:67-70), which throws `BESInternalFatalError` if the first container is not an NGAP container.

Where `alt_access()` gets the DMR++

`alt_access()` calls `get_dmrpp_from_cache_or_remote_source()` (:592-633), which tries these sources in order:

1. **The caches** (only if `d_use_dmrpp_cache`):
   - first the in-process `MemoryCache<string>`, keyed by real name;
   - then the shared on-disk `FileCache`, keyed by `hash_key(real_name)`. A file-cache hit is copied into the memory cache.
2. **The OPeNDAP bucket**, if `UseOPeNDAPBucket` is on and the real name starts with `source/`:
   - `build_dmrpp_url_to_owned_bucket()` turns `source/<host>/<key>` into `https://<host>/<key>[.dmrpp]`;
   - `curl::http_get()` fetches it, with S3 signing if credentials match;
   - the filter inserts `dmrpp:trust="true"` before `dmrpp:version`.

3. **The DAAC bucket**, if the real name starts with `collections/` or `providers/`:
   - `build_data_urls_to_daac_bucket()` asks CMR, through `NgapApi`, for the `(https URL, s3 URL, s3credentials URL)` triple. That answer is optionally memory-cached with the key `rest_path:uid`;
   - the code adds `.dmrpp`, primes `SignedUrlCache`, and tries a presigned S3 GET; if that fails, it falls back to an EDL-authenticated GET through TEA;
   - the filter replaces the `href="OPeNDAP_DMRpp_DATA_ACCESS_URL"` placeholders with the real `href`, `dmrpp:s3`, `dmrpp:s3credentials` and `dmrpp:trust="true"` values.
4. **The local file system**, if `EnableDMRppLocalFilesForTesting` is on: reads `<catalog root><rest_path>[.dmrpp]`.
5. Otherwise it throws `BESNotFoundError`.

An empty result throws `BESInternalError`. A successful fetch is written to the file cache (followed by a purge) and to the memory cache.

What happens after the fetch

`alt_access()` then:

- parses the DMR++ again with pugixml to pull out `dmrpp:href`, `dmrpp:s3` and `dmrpp:s3credentials` from the root element;
- primes `SignedUrlCache` with them;
- calls `set_attributes("as-string")`;
- returns the text.

Nothing is cached on the instance itself. Every call repeats the cache lookup and the pugixml parse.

`release()`

`release()` returns `true` and does nothing. The DMR++ is a string and there is nothing to unlock.

NGAP-specific findings

1. **The OPeNDAP-bucket path does not check AllowedHosts** (Verified by code trace).
   - `curl::http_get(const string&, ...)` (CurlUtils.cc:1287) never calls `AllowedHosts::is_allowed()`. Only the `shared_ptr<http::url>` entry points do (:1175, :1882).
   - A real name of `source/<any host>/<path>` makes the BES fetch `https://<any host>/<path>.dmrpp`, and the filter then marks it trusted.
   - **Exposure:** only when `NGAP.UseOPeNDAPBucket=true`. The code default is true; the shipped `ngap.conf` sets false. Make the code default match the config, or check the host.
2. **Out-of-bounds read** (Verified by code trace). For the real name `source/` (or `source//x`), `BESUtil::split()` skips empty tokens and returns `["source"]` (or `["source","x"]`), and `parts[1]` is then read past the end. This is client-controlled.
3. **The DMR++ cache is keyed by path only; the CMR cache is keyed by path plus UID** (Inferred).
   - On a DMR++ cache hit, no CMR query runs for that user.
   - If CMR visibility is meant to be per user, that check is skipped for any DMR++ another user already pulled into the cache.
   - The data reads still need that user's S3 or EDL access, so only the metadata is exposed.
4. **Wrong variable in a debug message** (Verified). `.cc:681` tests `s3_attr.empty()` a second time where it should test `s3credentials_attr`. Only debug output is affected.
5. **`"as-string"` leaks into the DMR** (Verified by code trace). `DmrppRequestHandler.cc:315-317` passes `get_attributes()` (`"as-string"`) to `dmr->set_filename()` and `dmr->set_name()`. The name is later overwritten from the DMR++ `<Dataset name=...>` (DMZ.cc:304), but the DMR's filename stays `"as-string"`. Low impact; the intent was presumably a flag, not a file name.
6. **Static state with no locking.** `MemoryCache` has no mutex, and the constructor rewrites the static flags. The code comments already call this "not thread-safe". It is safe under the BES model of one request at a time per process; it would not be with threads.
7. **Teardown uses the wrong store name** (Verified). `DmrppModule::terminate()` calls `deref_persistence(modname)` (`"dmrpp"`), but the store was added as `"ngap"` (DmrppModule.cc:75,93). The store survives until the list singleton is destroyed. The BES never calls `dlclose`, so the code is still mapped at that point. This is a correctness smell, not a crash.

The other module containers (summary)

All four follow one pattern:

- The real name is a URL or path.
- `access()` uses `http::RemoteResource` to download into `Http.RemoteResource.TmpDir` and returns the local temp-file name.
- `access()` sets the container type from what was downloaded.
- `release()` deletes the `RemoteResource`, which removes the temp file.

Container	Store name	`access()` returns	Notes
`S3Container`	`s3`	Temp file holding `<real_name>.dmrpp`, filtered	The DMR++ URL is built **trusted** from the client's real name (S3Container.cc:143-170), so AllowedHosts is skipped (AllowedHosts.cc:161). With `S3.inject_data_urls`, data hrefs are marked trusted too. `_duplicate` is correct. Holds a `shared_ptr`.

Container	Store name	`access()` returns	Notes
`GatewayContainer`	`gateway`	Temp file holding the remote file	The constructor checks AllowedHosts on the URL with credentials stripped. The fetch uses the original URL, credentials included.
`CmrContainer`	`CMR`	Temp file holding the granule	**Every** `access()` runs a live CMR query before it checks its cached resource, so the double `access()` costs two CMR round trips. Type is the placeholder `nc` until `access()`.
`HttpdCatalogContainer`	`RemoteResources`	Temp file holding the file	Rebuilds `HttpdCatalog` and re-reads keys on every `access()`. The collection path is not normalized for `..`, though the host is still checked.

`_duplicate` bug in Gateway, CMR and Httpd (Verified for Gateway at GatewayContainer.cc:100-108):

- The guard tests `copy_to.d_remoteResource` (the fresh target, always null) instead of `this->d_remoteResource`.
- It then copies the raw owning pointer.
- Duplicating an accessed container would double-free.
- These three use a non-virtual `_duplicate(Derived&)` overload rather than overriding the virtual one.

Container stores

BESContainerStorage (dispatch/BESContainerStorage.h)

This is the abstract store. It has a name (`_my_name`) and pure virtual methods:

- `look_for`;
- `add_container`, in two overloads: `(sym, real, type)`, marked `@deprecated` but used everywhere, and `(BESContainer*)`;
- `del_container`, `del_containers`;
- `isData`;
- `show_containers`;
- `dump`.

The only concrete code is `show_container()` (BESContainerStorage.cc:47-53), which writes one `<container name= type=>real</container>` tag.

The header comments describe MySQL or file-backed stores. Only the in-memory store and the test-only file store exist.

BESContainerStorageVolatile (dispatch/BESContainerStorageVolatile.{h,cc})

Every production store derives from this one. It holds `map<string, BESContainer*>` and owns the containers in it.

- **Constructor: requires `BES.Data.RootDirectory`** and throws `BESSyntaxUserError` if it is empty. It also reads `BES.FollowSymLinks`. Remote stores never use either value, but a `bes.conf` without `BES.Data.RootDirectory` cannot load the NGAP, S3, Gateway or other remote modules.
- **`look_for()`:** returns `ptr_duplicate()` of the prototype. The caller owns the copy.
- **`add_container(sym, real, type)`:**
  - throws if `type` is empty or `sym` already exists;
  - runs `BESUtil::check_path(real, root, follow)`;
  - builds a `BESFileContainer` with `assemblePath(root, real)`, and sets the relative name to `real`.
- **`add_container(BESContainer*)`** (protected): takes ownership on success. It checks the pointer, the type and duplicates, but does not validate the path. **If it throws, the passed container leaks**, because every subclass does `auto c = new X; add_container(c);`. In practice the caller's `setContainer` has already deleted any duplicate name, so this rarely fires.
- **`isData()`: leaks** the duplicate it gets from `look_for()` (:251). `BESContainerStorageFile::isData` deletes its copy; this one does not.
- **`show_containers()`:** strips `_root_dir` from each real name before reporting it.

BESFileContainerStorage (dispatch/BESFileContainerStorage.{h,cc})

This is the `"catalog"` store. Every file-based handler module registers it, with a ref-count guard: `dapreader`, `nc`, `h5`, `h4`, `ff`, `fits`, `gdal`, `csv`, `ncml` and `dmrpp`.

The header guard is still `BESContainerStorageCatalog_h_`, from an old class name.

- **Constructor:** looks up the `BESCatalog` of the same name and takes its `BESCatalogUtils`. It then **overwrites** `_root_dir` and `_follow_sym_links` with the catalog's `BES.Catalog.<n>.RootDirectory` and `FollowSymLinks`.
  - The value the base class required from `BES.Data.RootDirectory` is discarded.
  - If the catalog does not exist, `find_catalog()` returns null and the next line dereferences it. Modules avoid this by creating the catalog first.
- **`add_container()`** (:118-156):
  - strips trailing slashes and takes the basename;
  - throws `BESForbiddenError` unless `BESCatalogUtils::include()` accepts it (include and exclude regexes);
  - fills an empty `type` from TypeMatch (`get_handler_name(real_name)`);
  - passes the result to the volatile base, which does the path check and builds the `BESFileContainer`.

Undefined behavior: for an empty `real_name`, `stopat = size()-1` wraps to `npos`, and `real_name[npos]` is read. The same happens for an all-slash name, once the loop walks below index 0.

- **`isData()`:** overrides the leaky base. It uses TypeMatch only and asks `BESServiceRegistry` what the type provides.
- **`dump()`:** prints `utils: <name>` instead of anything about the utils object (cosmetic).

BESContainerStorageFile (dispatch/BESContainerStorageFile.{h,cc})

This store reads `BES.Container.Persistence.File.<name>`, a file of `sym real type` lines. Its `look_for()` returns a new `BESFileContainer` with no root and no path check, and both `add_container` overloads throw "not implemented". Only `pfileT` uses it.

The parser:

- reads into `char[80]`, so a line of 80 or more characters fails with a misleading "incomplete line" error;
- drops a final line that has no trailing newline;
- leaks the earlier entry when a symbolic name repeats.

Other stores

`NgapOwnedContainerStorage`, `S3ContainerStorage`, `GatewayContainerStorage`, `CmrContainerStorage` and `HttpdCatalogContainerStorage` each override `add_container(sym, real, type)` to call `new XContainer(...)` and then the protected `add_container(BESContainer*)`. Gateway's container constructor checks AllowedHosts; none of the others validate anything when the container is added.

BESContainerStorageList (dispatch/BESContainerStorageList.{h,cc})

This is a Meyers singleton (`TheList()`). It holds `vector<StorageEntry>`, where each entry is a `unique_ptr<BESContainerStorage>` plus a ref count, all guarded by a `recursive_mutex`.

Method	Behavior	Callers
`add_persistence(cp)`	Appends with count 1. **Returns false and does not take ownership** if the name already exists.	Module `initialize()`
`ref_persistence(name)`	Increments the count; returns false if the store is absent.	Module `initialize()`, as the guard before `add_persistence`
`deref_persistence(name)`	Decrements the count; at 0, erases the entry, **which deletes the store and all its prototype containers**.	Module `terminate()`
`find_persistence(name)`	Raw, non-owning pointer.	`setContainer`, `define` (with `space`), the delete handlers, `BESCatalogUtils::isData`, ncml `DDSLoader`
`look_for(sym)`	First store, in registration order, that returns non-null wins. Otherwise logs and throws `BESSyntaxUserError` unless `is_nice()`.	`define` without `space`
`delete_container(sym)`	Calls `del_container(sym)` on **every** store.	`setContainer` only
`show_containers(info)`	`<store name=...>` for each store.	`showContainers`
`is_nice()`	`BES.Container.Persistence` is `nice`, `Nice` or `NICE`.	`look_for` only

Observations

- **Ownership docs are wrong.** The `add_persistence` doc says "does NOT take ownership". In fact the list owns the store on success. On failure (a duplicate name) the caller still owns `cp`.
  - The `nc` and `h5` modules check `ref_persistence` first.
  - `DmrppModule` ("ngap"), `GatewayModule` and `S3Module` do not, so a double registration would leak the new store (Inferred).
- **`is_nice()` is documented in the header as "unclear".** It is described correctly in `docs/BES_Configuration.md` (nice means log, strict means throw), and `bes.conf.in:173` sets `strict`.
  - Nice mode makes little difference: `BESXMLDefineCommand::prep_request` throws anyway when `look_for()` returns null.
- **Registration order decides `look_for(sym)`.** It searches the stores in order, but `setContainer` first removes `sym` from every store, so in practice only one store holds a given symbolic name.
- **`StorageEntry::get_name()`** carries leftover "hypothetical method" comments from the Gemini C++14 rewrite.

How a request uses containers

```
<setContainer name="c" space="ngap">collections/.../granule.nc</setContainer>
<define name="d" space="default"><container name="c"/></define>
<get type="dap" definition="d" returnAs="dmrpp"/>
```

1. **setContainer**, through `BESXMLSetContainerCommand`, then `BESSetContainerResponseHandler::execute()` (:77-97):
   - `space` defaults to `"catalog"`, not `"volatile"`. No volatile store is registered in production, and the creation in `BESDefaultModule.cc:99-100` is commented out.
   - If the path starts with a catalog name, that name replaces `space`.
   - It calls `TheList()->delete_container(sym)`, which removes `sym` from all stores.
   - It then calls `find_persistence(space)`, then `del_container(sym)` again (redundant), then `add_container(sym, real, type)`.
   - The store builds the prototype. **For NGAP, this is where the constructor and the fd leak run.**
2. **define**, through `BESXMLDefineCommand::prep_request()` (:340-381):
   - Calls `find_persistence(space)->look_for(c)`, or `TheList()->look_for(c)` when there is no `space`, and gets a **duplicate**.
   - Sets the constraint, the DAP4 constraint and function (falling back to the `<define>`-level defaults) and the attributes.
   - Pushes the duplicate into the command's DHI.
   - `BESDefineResponseHandler` moves the pointers into a new `BESDefine`, which owns them (`~BESDefine` deletes them), and stores it in the `"default"` definition store, replacing any old definition of that name.
3. **get**, through `BESXMLGetCommand::prep_request()`: copies the `BESDefine`'s container pointers, *borrowed*, into its DHI.
4. **Execute.** The DAP response handlers call `BESRequestHandlerList::execute_each()`, which for each container calls `execute_current()` (:227-264):
   - `container->access()`: for NGAP this sets the type to `dmrpp`;
   - `find_handler(get_container_type())`, then `find_method(action)`, then call the method.
   - Most handlers call `dhi.container->access()` **again** to get the file name: netCDF, HDF4, HDF5, GDAL, CSV, FITS, FreeForm, NcML, dapreader and the dmrpp non-NGAP path.
   - The dmrpp handler `dynamic_cast`s to `NgapOwnedContainer` and calls `alt_access()` instead.
   - In `dap/BESDataResponseHandler.cc:97-101` and `BESDap4ResponseHandler.cc:78-82`, if the metadata store holds a DMR++ for the dataset, the container type is changed to `dmrpp` and the attributes are set to `MDS_HAS_DMRPP`. Nothing reads that attribute value, and the write overwrites any user-supplied `<attributes>`.
5. **End.** `BESInterface::finish()` calls `end_request()` (BESInterface.cc:460-468), which calls `release()` on every container in `d_dhi_ptr`. That is the DHI of the last command executed, normally the `get`.
   - Containers are **not deleted** at the end of the request.
   - The `BESDefine` and its duplicates live until a later `define` with the same name, a `deleteDefinition`, or process exit.
   - The prototypes live in their store until the next `setContainer` of that name.

Consequences

- Each `beslistener` child keeps one prototype per symbolic name and one `BESDefine` per definition name. The OLFS reuses names such as `d1` and `ngapContainer`, so memory does not grow per request. Resources acquired in `access()` and not released in `release()` do accumulate.
- **`release()` coverage depends on the command order.** It is called only for the DHI that `d_dhi_ptr` last pointed to. A `BESFileContainer` that was accessed under some other command's DHI keeps its uncompress-cache read lock until the next `access()` or `release()` on it (Inferred). The destructor does not call `release()`.
- **`access()` runs twice per request by design.** Remote containers that do real work in `access()` have to cache the result on the instance (S3, Gateway, Httpd do) or make it trivial (NGAP). `CmrContainer` re-queries CMR on each call.

Where containers touch the framework

Area	File(s)	Interaction
XML commands	`xmlcommand/BESXML{SetContainer,Define,Get,DeleteContainer(s),Show}Command.cc`	Parse store names, symbolic names, constraints, attributes
Response handlers	`dispatch/BES{SetContainer,Define,DelContainer(s),ShowContainers}ResponseHandler.cc`	Talk to `BESContainerStorageList`
Definitions	`dispatch/BESDefine.cc`, `BESDefinitionStorageVolatile.cc`	Own the duplicated containers
Data handler interface	`dispatch/BESDataHandlerInterface.h`	`containers`, `container`, `first_container()`, `next_container()`; borrowed pointers; `clean()` never deletes them
Dispatch	`dispatch/BESRequestHandlerList.cc`	`access()`, then type, then handler
Request end	`dispatch/BESInterface.cc`	`release()`
DAP	`dap/BESDapResponse.cc`, `BESConstraintFuncs.cc`, `BESDapTransmit.cc`, `BESDapResponseBuilder.cc`	Read the constraint, real name and type; re-dispatch by type
Catalog	`dispatch/BESCatalogUtils.cc:471` (`isData`), `BESCatalogList`	`find_persistence` to classify catalog nodes
NcML	`modules/ncml_module/DDSLoader.cc`	Builds its own containers in the `catalog` or `gateway` store,

Area	File(s)	Interaction
		calls `execute_current`, then `release` and `delete`
Module lifecycle	each `*Module.cc` `initialize()`/`terminate()`	`ref`/`add`/`deref_persistence`

Tests

- **Unit tests in `dispatch/unit-tests`:**
  - `containerT`: the strict and nice `look_for` behavior, and compressed files;
  - `pvolT`: the volatile store;
  - `pfileT`: the file store;
  - `plistT`: the list;
  - `reqhandlerT`, `constraintT`, `defT`.
- **NGAP unit tests:** `NgapOwnedContainerTest` covers URL building, the caches, `access()` and `alt_access()`. Some tests need network access or credentials.
- **Not covered:**
  - `delete_container` across stores;
  - the `BESDefine` ownership chain;
  - `release()` coverage;
  - repeated `NgapOwnedContainer` construction, which would have caught the fd leak and the size compounding;
  - `isData` on volatile stores;
  - edge-case real names (empty, `source/`).

Stale or wrong documentation

Where	What it says	What the code does
`NgapOwnedContainer.h:55-59`	`access()` returns DMR++ XML	`access()` returns `""`; `alt_access()` returns the XML
`BESContainerStorageList.cc:62`	`add_persistence` does not take ownership	It takes ownership on success
`BESContainerStorageList.h:168`, `.cc:155`	`is_nice` purpose unclear	Controls whether `look_for` throws or logs (`BES.Container.Persistence`)
`BESSetContainerResponseHandler.cc:60-62`, `docs/BES_Creating_New_Modules.md`	Default store is "volatile"	Default is `"catalog"`; no volatile store exists
`BESRequestHandlerList.cc:235-238`	`access()` changes the extension so the type is right	`BESFileContainer::access()` never changes the type
`BESFileContainer.cc:63`	`'../' not allowed`	Any `..` is rejected
`ngap.conf.in:9-16`	`NGAP.DMRppCacheSize`, `NGAP.CMRCacheSize`	Keys read are `...Size.Items` / `...Purge.Items`

Suggested fixes, smallest first

1. `FileCache::initialize()`: close `d_cache_info_fd` if it is already open. Separately, move the NGAP static-cache setup and key reads into a run-once path, such as `DmrppModule::initialize()` or a static flag. This fixes the fd leak and the size compounding.
2. `NgapOwnedContainer::build_dmrpp_url_to_owned_bucket()`: throw `BESSyntaxUserError` when `parts.size() < 2`.
3. `BESContainerStorageVolatile::isData()`: delete the duplicate, or wrap it in `unique_ptr`.
4. `BESFileContainerStorage::add_container()`: reject an empty or all-slash `real_name` before the loop.
5. `Gateway`/`Cmr`/`HttpdCatalogContainer::_duplicate`: test `this->d_remoteResource`.
6. `DmrppModule::terminate()`: `deref_persistence("ngap")`.
7. `.cc:681`: test `s3credentials_attr`.
8. Security:
   - check AllowedHosts on the `source/` DMR++ URL, or set the code default of `NGAP.UseOPeNDAPBucket` to false;
   - do not mark the client-supplied S3 URL as trusted.

   These change runtime behavior and need a decision about which deployments rely on the current behavior.

Validation

- I read every file named in this document. Two read-only subagents surveyed the module containers and the framework call sites. I checked their key claims against the code; the two corrections are in the work log.
- I ran `dispatch/unit-tests/containerT`: OK, 7 tests.
- I ran `modules/dmrpp_module/ngap_container/unit-tests/NgapOwnedContainerTest`: OK, 30 tests. Both used the existing in-tree `--enable-developer` build on macOS arm64.
- The fd-leak probe was a scratch program in the session scratchpad, linked against `dispatch/.libs/libbes_dispatch`. It is not in the repository.
- Not run: the integration test suites, any NGAP request against live NASA services, and any Linux test.